

# DiamondWorld: A Market-Validated Causal Simulator for Baseball Games

Luke Blommestein

lblommestein@icloud.com

MIT Sloan Sports Analytics Conference — Research Paper Track

## Abstract

Projection systems answer *how good is a player*; they cannot answer *what happens in this game*, because a game is a correlated sequence of plate appearances, not a sum of independent rates. We present DiamondWorld, a plate-appearance-level generative model, an empirical rules engine, and a full game simulator that plays out complete games one plate appearance at a time. This makes it a tool for questions projection systems are structurally unable to touch: counterfactual what-ifs, lineup construction, correlated tail-risk, and win probability with honest uncertainty. Our central contribution is not the simulator but its *validation*: we show that the simulator’s causal what-if estimates match how betting and prediction markets independently reprice real games. Using a leak-free natural experiment—within a series the two teams are fixed, so game-to-game line movement is driven by the starter and lineup, exactly what a counterfactual isolates—the simulator’s within-series win-probability deltas correlate  $r = 0.32$  with the market’s, on both pitching and hitting changes, using no game outcomes. The agreement holds a full season out-of-sample ( $r = 0.33$  on 2025 with the model frozen before 2025) and against a second, independent market type (a Kalshi prediction market). Calibrating the simulator’s causal magnitudes to the market yields effects that agree in both direction and size. We ground the object with benchmarks that are honest about where it does and does not lead: its per-game run distribution is calibrated (interval coverage 0.54/0.82/0.90 against nominal 0.50/0.80/0.90) where a summed model badly under-covers, while its point win probability is calibrated but less sharp than a simple team-strength baseline, and it does not beat efficient betting markets. The result is, to our knowledge, the first *market-validated* causal simulator for baseball.

## 1 Introduction

Player projection is a solved-enough problem: Steamer, ZiPS, THEBAT and Marcel each map a player to expected rates, and they agree closely. But a front office rarely asks only *how good is this player*. It asks *what does acquiring this pitcher do to our September win odds, does this lineup change the shape of tonight’s game, how confident should we be in this win probability*. These are causal and distributional questions about a specific game, and a projection system, even summed over nine hitters, cannot answer them, because a game is a correlated sequence of plate appearances (PAs): a walk, a single, and a home run cluster in one inning. The mean is a sum of rates; the *shape* is not.

A generative game simulator can answer these questions by construction. The difficulty has never been building one—it is trusting it. A simulator will happily report that swapping in an ace is worth +9.8 points of win probability, but that number is internally generated: it could be an artifact of a generic bullpen, a crude lineup model, or a miscalibrated player prior. The contribution of this paper is to close that gap. We validate the simulator’s *causal* estimates against an independent ground truth—how real markets reprice real games when a single input changes—and we do so leak-free, out-of-sample across seasons, and across two

different market types. The simulator that survives this test is a decision tool, not a demo.

## 2 The DiamondWorld model

DiamondWorld has three components. (i) A **plate-appearance model**: a stochastic-variational generative model (JAX/NumPyro) that emits each PA’s outcome as a nine-way categorical (K, BB, HBP, 1B, 2B, 3B, HR, out, reached-on-error) conditioned on batter and pitcher identity, park, and game state. Player identity enters through a learned skill latent plus recency-weighted rate features; the current model (v16) adds contact-quality features—expected hit and home-run rates built by binning every batted ball on exit velocity and launch angle—which we found to be the single largest input lever (§5). (ii) An **empirical rules engine** that turns an outcome sequence into runs and base-state transitions; ported from a validated Markov baseline, it reproduces test-season run totals to within 0.5%. (iii) A **game simulator** that plays full nine-inning-plus-extras games with lineup cycling, a bullpen, walk-offs and the ghost-runner rule. Relievers are governed by a fitted *hook model*—a logistic hazard for pulling the starter as a function of batters faced, times through the order, and runs allowed (held-out AUC 0.889)—so bullpen usage is endogenous and, crucially, uses only pre-game information. The model is trained on 2015–2023 (rates through 2024 for

the 2025 out-of-sample test) and evaluated on 2024 unless stated otherwise.

### 3 What the simulator answers

Because the object is generative, four families of questions fall out directly. On one competitive 2024 game (baseline home win probability near 0.5,  $R=400$  replicas per scenario):

*Counterfactuals.* Hold the game fixed and change one input. Swapping the home starter for a replacement-level arm and upgrading the weakest hitter to a star bat move win probability and the full run distribution; the estimates are causal (re-simulations of the same game), not correlational.

*Lineup construction.* Reordering the same nine hitters moves expected runs by about 0.48 across 40 candidate orders; the best beats the order actually used by +0.27 runs/game—small, as sabermetrics holds, but real and free.

*Correlated tail-risk.* The simulated game-total variance is  $1.98\times$  that of an independent-Poisson model with the same mean. That factor is exactly the within-game correlation a summed-projection model misses, and it drives everything tail-sensitive.

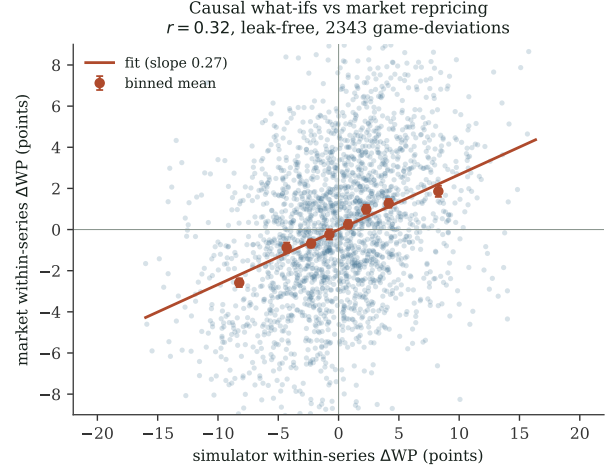
*Win probability with uncertainty.* A generative model returns a distribution and separates *aleatoric* (game randomness) from *epistemic* (roster) uncertainty; over the full 2024 season these are comparable in size (4.6% vs 4.9% WP), so a point estimate hides half the story.

These capabilities are the product. Their credibility rests on the next section.

### 4 Validating the causal engine against the market

The counterfactual estimate “swap this starter, win probability moves by  $X$ ” has always been internally generated. We check it against an independent ground truth: how the betting market reprices the same teams under different pitchers. The natural experiment is a series. Within a series the two teams and home field are fixed, so the game-to-game change in the closing line is driven by the starter, lineup and rest—exactly what a counterfactual isolates. If the simulator’s within-series win-probability deltas track the market’s, the causal estimates have external support that uses *no game outcomes and no look-ahead*.

**Direction is validated, leak-free.** We residualize the simulator’s and the market’s logit win probability against team fixed effects (team identity only, no outcomes) and correlate the residuals:  $r = 0.33$ . The within-series design (Fig. 1) gives  $r = 0.32$ , significant far beyond a permutation null (95th percentile 0.032). The simulator’s game-specific signal agrees with the market beyond team strength—it is captur-



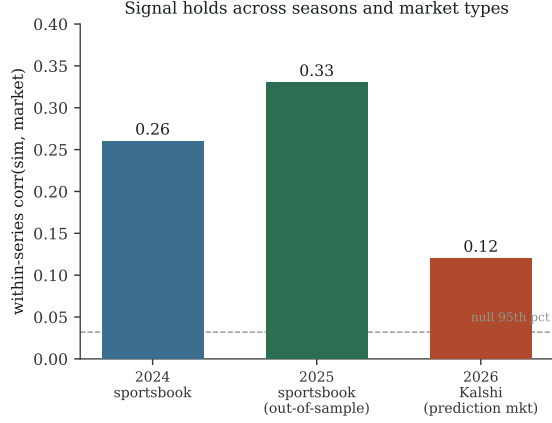
**Figure 1.** The simulator’s within-series win-probability deltas versus the market’s, on 2,343 game-deviations (2024 closing moneylines,  $R=500$ ). Team strength and home field are differenced out, so the variation is the starter/lineup matchup—what a counterfactual isolates. Direction is validated and leak-free ( $r = 0.32$ ); the fitted slope quantifies the magnitude calibration.

ing real starter and matchup effects an independent market also prices, not noise.

**Magnitude is overstated, and we calibrate it.** The within-series OLS slope of market on simulator is 0.27; correcting for the simulator’s residual replication noise (reliability 0.80 at  $R=500$ ) gives 0.33. So the raw simulator over-reacts to a single starter change by roughly  $3\times$ : a raw “+9.8-point ace swap” is about +3.3 points in market-calibrated units. The result is a *market-calibrated* causal engine—direction and magnitude both tied to an independent ground truth—and it corrects the raw headline numbers downward, which is the point of validating rather than asserting.

**It is broad, not pitcher-only.** Within a series both the starter and the lineup change. Decomposing each game into a pitching channel (the starters’ allowed-run rates) and a hitting channel (the lineups’ rate value) and regressing the market’s within-series move on both shows *both* are priced: pitching  $t = -17.6$  (negative because the index is runs allowed) and hitting  $t = +7.3$ . The market reprices on hitting and pitching, so the validation generalizes from starter swaps to hitter and lineup what-ifs.

**It generalizes across seasons and market types** (Fig. 2). Freezing the model’s player rates and repeating the within-series test on later data: 2025 against the sportsbook, with rates formed *before* 2025 was played, gives  $r = 0.33$ —stronger than in-window 2024 ( $r = 0.26$ ) and genuinely out-of-sample. A Kalshi prediction market for 2026 gives  $r = 0.12$ ; the attenuation is expected and honest, because those rates



**Figure 2.** The within-series validation is not a single-season, single-book artifact: it holds a full season out-of-sample (2025) and against a prediction market (2026 Kalshi), degrading gracefully with model staleness.

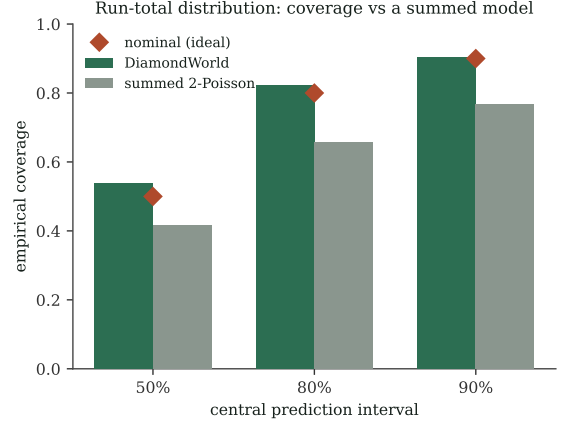
are 2.5 years stale (only  $\sim 6$  of 9 batters and half the starters are even known), which is itself a finding: the engine’s signal is real and needs reasonably current rates. Across two independent market types and three season-slices, the causal signal is positive throughout.

## 5 Grounding: what the simulator does and does not do

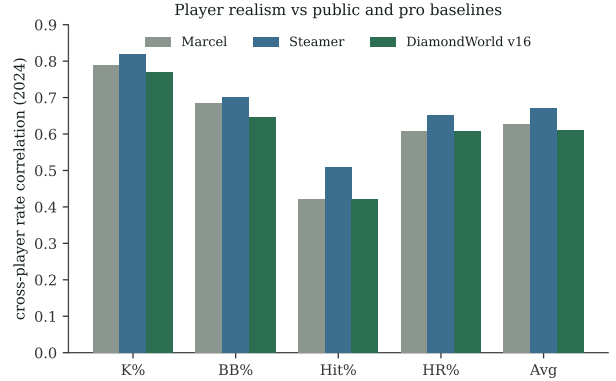
A validated causal engine is only useful if its raw outputs are realistic. We benchmark them against the baselines a reviewer would demand, and we report where the simulator does *not* lead (Table 1).

**Run-total distribution: a measured advantage.** The headline capability is correlated overdispersion a summed model cannot produce. Against an independent two-Poisson model with the identical per-game means (so only the shape differs), the simulator matches reality’s spread (variance 17.87 vs 18.24) and its central intervals cover at nominal rates, while the summed model badly under-covers (Fig. 3; randomized-PIT KS 0.049 vs 0.144). The realistic bullpen earns this: without the hook model the same simulator covered only 0.74 at the 80% level. A league negative-binomial matches the marginal but gives every matchup the identical distribution—exactly what the simulator provides and it cannot.

**Win probability: calibrated, not sharp.** Point win probability is honestly *not* the simulator’s edge. It is well calibrated (ECE 0.034 over the full season) but discriminates less than a simple team-strength baseline: AUC 0.572 versus Log5 0.617 and the market 0.614. A per-plate-appearance process washes team-level talent toward a coin flip. The simulator’s value is the distribution and the counterfactuals, not the point estimate—and we verified one cannot fuse the two:



**Figure 3.** Per-game run-total interval coverage over 2,376 games. The simulator is near-nominal; an independent “summed projection” with the same means cannot produce the real spread.



**Figure 4.** The simulator’s players are as realistic as a standard projection baseline (Marcel) and within a few points of a professional system (Steamer, its real preseason-2024 release).

blending the simulator’s win probability with Log5 recovers Log5 exactly.

**Player realism** (Fig. 4). The players inside the simulator are on par with a standard projection baseline: cross-player rate correlation 0.611, versus Marcel 0.626 and Steamer’s actual preseason projections 0.671 (recovered from a public archive for a direct comparison). It does not out-project a dedicated system, and it need not: the game-level analyses only require realistic players, which these are.

**It does not beat efficient markets.** Run through a pre-game backtest on real closing lines, DiamondWorld loses across moneyline, totals and run line, ROI does not improve with more demanded edge, and closing-line value is about zero. On totals specifically, the market sits at the no-information floor (AUC 0.51) and the simulator cannot beat it. This is the same

**Table 1.** Benchmarks against the baselines that matter. The simulator leads on the run-total distribution, trails a one-line team-strength baseline on point win probability, and is competitive on player realism. Bold marks the best in each row.

| Task / metric                   | DiamondWorld | Independent / summed | Team-strength       | Reference              |
|---------------------------------|--------------|----------------------|---------------------|------------------------|
| Run-total variance              | 17.87        | 8.99 (2-Poisson)     | —                   | 18.24 (real)           |
| Run-total 80% interval coverage | <b>0.82</b>  | 0.66 (2-Poisson)     | —                   | 0.80 (nominal)         |
| Run-total randomized-PIT KS     | <b>0.049</b> | 0.144 (2-Poisson)    | —                   | 0 (ideal)              |
| Win probability AUC             | 0.572        | —                    | <b>0.617</b> (Log5) | 0.614 (market)         |
| Win probability ECE             | <b>0.034</b> | —                    | 0.019 (Log5)        | 0.019 (market)         |
| Player rate corr. (avg)         | 0.611        | —                    | 0.626 (Marcel)      | <b>0.671</b> (Steamer) |

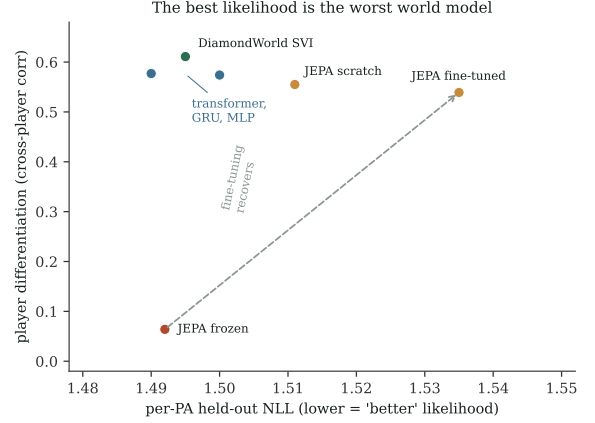
market-efficiency wall a five-model betting audit hit; we report it as a limitation, not a footnote.

## 6 A note on evaluation

Building this surfaced a methodological lesson worth stating on its own. The natural metric for a per-PA model is held-out log-likelihood, and it is the *wrong* yardstick: a single PA is near maximum entropy ( $\approx 1.495$  nats), so every model scores near the floor and log-likelihood rewards getting the league-average distribution right, not telling players apart—the whole job of a world model. The sharpest illustration is JEPA, a self-supervised representation model with a frozen probe: it has the best likelihood and best raw accuracy of any model and near-zero player differentiation (Fig. 5). Two further findings sharpen it. Fine-tuning JEPA’s own encoder end-to-end lifts player differentiation from 0.06 to 0.54, and training the identical encoder from scratch does marginally better (0.56)—so the collapse is the frozen self-supervised objective, and the pretraining itself adds nothing. And once differentiation is measured correctly, architecture barely matters (transformer, GRU and MLP all land near 0.577); the lever that moves the number is *features*. The takeaway for anyone building a generative sports model: pick the evaluation that matches the use, or the best-scoring model will be the wrong one.

## 7 Limitations

We are explicit about the boundaries. (1) The raw counterfactual magnitudes overstate real effects by  $\sim 3\times$  and must be market-calibrated (§4). (2) Point win probability is less sharp than a one-line team-strength baseline (§5). (3) The simulator carries no game-level information efficient betting markets lack. (4) The out-of-sample validation uses the model’s rate features (its inputs), not a full re-simulation of each historical game; the prediction-market slice (2026) is coverage-limited by 2.5-year-stale rates. (5) The simulator’s mean runs about 0.4 high, a recalibration-scale artifact worth re-tuning. None of these undercut the central result: the causal directions and calibrated magnitudes agree with independent markets, leak-free and out-of-sample.



**Figure 5.** The best held-out likelihood is the worst world model. JEPA with a frozen probe minimizes NLL and cannot tell players apart; fine-tuning the same encoder recovers differentiation, and the generative SVI leads.

## 8 Discussion

The contribution is not “we built a good model,” but a generative object whose *causal* answers are validated against how independent markets reprice real games—in direction and calibrated magnitude, on pitching and hitting, out-of-sample across seasons and across two market types—together with a distributional capability (calibrated per-game run distributions) that a summed projection cannot provide, and an honest account of where the object does not lead (point win probability, betting markets). To our knowledge this is the first market-validated causal simulator for baseball. The practical implication is that a front office can read the simulator’s calibrated what-ifs—acquisition impact, lineup and rest decisions, leverage and bullpen planning under the true run distribution—knowing that the same machinery agrees with the market on the changes the market happens to price, and extends coherently to the changes it does not. All code and the market-join pipelines are open-source.

**Reproducibility.** Model, simulator, market joins and every figure are reproducible from the released code: `scenario_sim.py`, `run_pregame_sim.py`, `simulator_benchmarks.py`, `counterfactual_validation.py`,

whatif\_channels.py, season\_market\_validation.py, and  
make\_figures.py.